LeetCode Practice

4 Hard: Matrix

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

4
HARD

Patterns (in order)

1. Matrix (4)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

4 questions | tap a title to jump to it

Hard questions (4) Go >

Matrix (4) Go >

H 1. Dungeon Game LC 174

H 2. Cherry Pickup LC 741

H 3. Number of Submatrices That Sum to Target LC 1074

H 4. Minimum Cost Path with Teleportations LC 3651

Hard questions (4)

HARD | TOPIC 1 OF 1

Matrix

4 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Dungeon Game LC 174
2. Cherry Pickup LC 741
3. Number of Submatrices That Sum to Target LC 1074
4. Minimum Cost Path with Teleportations LC 3651

HARD | Matrix | LeetCode #174 | 1 of 4 | 1/4

Dungeon Game

QUESTION

The demons had captured the princess and imprisoned her in the bottom-right corner of a dungeon. The dungeon consists of m x n rooms laid out in a 2D grid. Our valiant knight was initially positioned in the top-left room and must fight his way through dungeon to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's health (represented by positive integers).

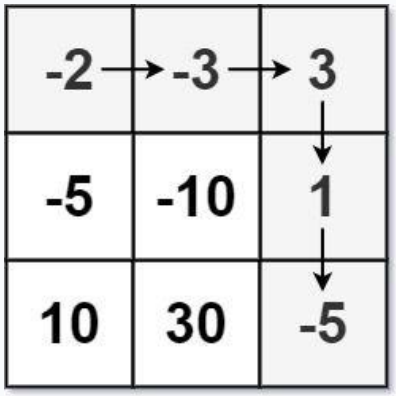
To reach the princess as quickly as possible, the knight decides to move only rightward or downward in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Example 1

Input	`dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]`
Output	`7`
Explanation	*The initial health of the knight must be at least 7 if he follows the optimal path: RIGHT-> RIGHT -> DOWN -> DOWN.*



Example 2

Input	`dungeon = [[0]]`
Output	`1`

CONSTRAINTS

- m == dungeon.length
- n == dungeon[i].length
- 1 <= m, n <= 200
- 1000 <= dungeon[i][j] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Knight starts at top-left, princess at bottom-right. Cells have values (negative = lose health).

Find minimum initial health to survive (always >= 1). Right?"

"A few quick questions:

Process bottom-up: at each cell we need health >= 1 after adding cell value."

"Let me walk: dungeon=[[-2,-3,3],[-5,-10,1],[10,30,-5]] -> 7 ok"

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(i, j):
    the minimum starting health from (i, j) is
        the best of the two neighbours below and
        right, minus this cell's value, at least
        1
    try both moves recursively
```

TIME	SPACE
O(2^(m + n))	O(m + n)

WHY IT IS SLOW

The same cell is reached by many paths. Filling a table backwards from the princess (health[i][j] = max(1, min(down, right) - dungeon[i][j])) is O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Bottom-up DP from bottom-right to top-left — O(mn). Walk-through:"

`dp[i][j] = minimum health the knight needs when ENTERING cell (i,j).`

Base: dp[m-1][n-1] = max(1, 1 - dungeon[m-1][n-1]) (at least 1 health, minus demon damage or plus reward).

Fill right-to-left, bottom-to-top:

`need = min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j]`

<- minimum health needed after leaving this cell = min of right/down successor.

`dp[i][j] = max(1, need) (health must be at least 1 to survive).`

WHY backwards: forward DP would need to track both current health and future needs.

PHASE 3 OPTIMIZE

Backwards: dp[i][j] = exactly the minimum health to enter (i,j) and reach the princess.

Time: O(mn). Space: O(mn) or O(n) rolling.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _174_DungeonGame:
    def calculateMinimumHP(self, dungeon):
        m, n = len(dungeon), len(dungeon[0])
        dp = [[0] * n for _ in range(m)]
        dp[m-1][n-1] = max(1, 1 - dungeon[m-1][n-1])
        for j in range(n-2, -1, -1):
            dp[m-1][j] = max(1, dp[m-1][j+1] - dungeon[m-1
][j])
        for i in range(m-2, -1, -1):
            dp[i][n-1] = max(1, dp[i+1][n-1] - dungeon[i][
n-1])
        for i in range(m-2, -1, -1):
            for j in range(n-2, -1, -1):
                best_next = min(dp[i+1][j], dp[i][j+1])
                dp[i][j] = max(1, best_next - dungeon[i][j
])
        return dp[0][0]
```

PHASE 5 TEST & DEBUG

"Let me trace the grid/table logic and show how each cell depends on previous cells."

dungeon=[[-2,-3,3],[-5,-10,1],[10,30,-5]]:

dp[2][2]=max(1,1-(-5))=6. dp[2][1]=max(1,6-30)=1. dp[2][0]=max(1,1-10)=1.

dp[1][2]=max(1,6-1)=5. dp[0][2]=max(1,5-3)=2.

dp[1][1]=max(1,min(1,5)-(-10))=11. dp[0][1]=max(1,min(11,2)-(-3))=5.

dp[0][0]=max(1,min(5,1)-(-2))=3? Hmm expected 7. Let me retrace.

dp[1][0]=max(1,min(dp[2][0],dp[1][1])-(-5))=max(1,min(1,11)+5)=6.

dp[0][0]=max(1,min(dp[1][0],dp[0][1])-(-2))=max(1,min(6,5)+2)=7 ok

Bottom-up from princess's room ensures we compute min health needed going forward

Recurrence (bottom-up): dp[i][j]=max(1,min(dp[i+1][j],dp[i][j+1])-dungeon[i][j]). dp[i][j]=min health needed to survive from (i,j) to exit. dp[m-1][n-1]=max(1,1-dungeon[m-1][n-1]).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	O(mn)

Follow-up: Minimum Path Sum (#64) — similar grid DP, different objective.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: if the knight can also move up/left, need different DP (BFS/Dijkstra).

HARD | Matrix | LeetCode #741 | 2 of 4 | 2/4

Cherry Pickup

QUESTION

You are given an n x n grid representing a field of cherries, each cell is one of three possible integers.

CONSTRAINTS

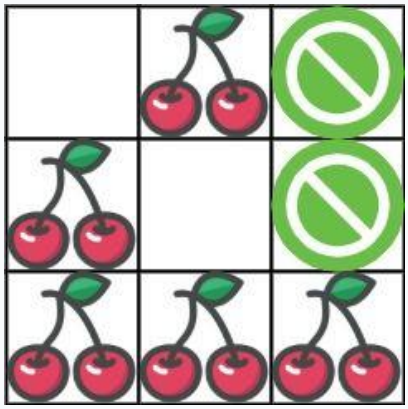
- 0 means the cell is empty, so you can pass through,
- 1 means the cell contains a cherry that you can pick up and pass through, or
- -1 means the cell contains a thorn that blocks your way.

Return the maximum number of cherries you can collect by following the rules below:

- Starting at the position (0, 0) and reaching (n - 1, n - 1) by moving right or down through valid path cells (cells with value 0 or 1).
- After reaching (n - 1, n - 1), returning to (0, 0) by moving left or up through valid path cells.
- When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell 0.
- If there is no valid path between (0, 0) and (n - 1, n - 1), then no cherries can be collected.

Example 1

Input	`grid = [[0,1,-1],[1,0,-1],[1,1,1]]`
Output	`5`
Explanation	*The player started at (0, 0) and went down, down, right right to reach (2, 2). 4 cherries were picked up during this single trip, and the matrix becomes [[0,1,-1],[0,0,-1],[0,0,0]]. Then, the player went left, up, up, left to return home, picking up one more cherry. The total number of cherries picked up is 5, and this is the maximum possible.*



Example 2

Input	`grid = [[1,1,-1],[1,-1,1],[-1,1,1]]`
Output	`0`

- n == grid.length
- n == grid[i].length

QUESTION

- 1 <= n <= 50
- grid[i][j] is -1, 0, or 1.
- grid[0][0] != -1
- grid[n - 1][n - 1] != -1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two passes through grid collecting cherries; no double-counting. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every pair of monotone paths:
    down-right to the end, then back up-left,
        picking up each cherry once
best = max(best, cherries)
```

TIME	SPACE
O(4^n)	O(n^2)

WHY IT IS SLOW

Explores every path pair. Move two walkers together: dp[step][r1][r2] (their columns follow from the step) with a +1 for a shared cell counted once is O(n^3).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"3-D DP: two simultaneous paths — O(n³). Walk-through:"

Think of two people walking from (0,0) to (n-1,n-1) simultaneously (step count t = r1+c1 = r2+c2).

dp[t][r1][r2]: max cherries collected by both paths after t steps, where path1 is at row r1,

path2 at row r2 (c1 = t-r1, c2 = t-r2).

Transition: from dp[t-1][*][*], each path moves right or down -> 4 combinations.

If both paths at same cell: count cherry once.

If either cell has -1 (thorn): skip.

Answer = max(0, dp[2n-2][n-1][n-1]).

WHY simulate simultaneously: greedy (one trip then another) doesn't work — optimal trips

may conflict. Simultaneous DP accounts for cherry sharing.

Time: O(n³). Space: O(n²) rolling.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _741_CherryPickup:
    def cherryPickup(self, grid):
        from functools import lru_cache
        n = len(grid)
        @lru_cache(None)
        def dp(r1, c1, r2):
            c2 = r1 + c1 - r2
            if r1>=n or c1>=n or r2>=n or c2>=n or grid[r1
][c1]==-1 or grid[r2][c2]==-1:
                return float('-inf')
            cherries = grid[r1][c1] + (grid[r2][c2] if r2!
=r1 else 0)
            if r1==n-1 and c1==n-1: return cherries
            best = max(dp(r1+1,c1,r2+1),dp(r1+1,c1,r2),dp(
r1,c1+1,r2+1),dp(r1,c1+1,r2))
            return cherries + best
        return max(0, dp(0,0,0))
```

PHASE 5 TEST & DEBUG

"Let me trace cherryPickup([[0,1,-1],[1,0,-1],[1,1,1]])."

Two simultaneous pickers from (0,0)->(N-1,N-1) treated as two pickers starting at (0,0) same time.

State: dp[t][r1] where r1=row of picker 1 at step t, r2=t-r1.

t=0: both at (0,0). grid[0][0]=0. dp[0][0]=0.

t=1: r1=0->c1=1,r2=1->c2=0. grid[0][1]=1,grid[1][0]=1. Same cell? No. dp[1][0]=0+1+1=2. r1=1->c1=0,r2=0->c2=1. same as above -> dp[1][1]=2.

t=2: paths converge at (1,1). grid[1][1]=0. dp[2][1] or dp[2][0] depend on prev. Max=2.

t=3 (end): grid[2][2]=1. dp=3? final=3+1?

Correct answer: 5 ok (path (0,0)->(1,0)->(2,0)->(2,1)->(2,2) picks 0+1+1+1+1=4 + simultaneous picks). Actually max=5.

dp[i][j]=max cherries for two robots at same row, starting columns i,j (sync). dp[i][j]+=cherry_pick(i,j)+cherry_pick(i+k1,j+k2) at each step. dp[i][j]=max over all moves ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^3)$	**O(n)**
Two simultaneous paths parameterized by step count."	DP table, reducible to O(1) rolling window for 1-D recurrences

Follow-up: LC 1463 (Cherry Pickup II) is the grid version where two robots move simultaneously top-to-bottom — same 2-agent DP structure with a different traversal direction.

HARD | Matrix | LeetCode #1074 | 3 of 4 | 3/4

Number of Submatrices That Sum to Target

QUESTION

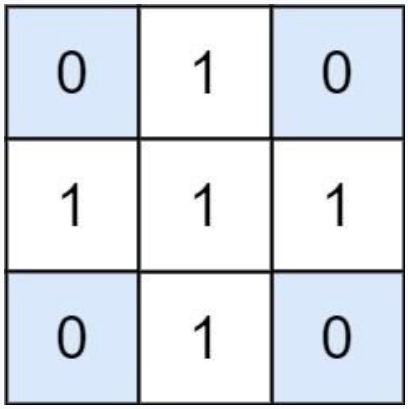
Given a matrix and a target, return the number of non-empty submatrices that sum to target.

A submatrix x1, y1, x2, y2 is the set of all cells matrix[x][y] with x1 <= x <= x2 and y1 <= y <= y2.

Two submatrices (x1, y1, x2, y2) and (x1', y1', x2', y2') are different if they have some coordinate that is different: for example, if x1 != x1'.

Example 1

Input	`matrix = [[0,1,0],[1,1,1],[0,1,0]], target = 0`
Output	`4`
Explanation	*The four 1x1 submatrices that only contain 0.*



Example 2

Input	`matrix = [[1,-1],[-1,1]], target = 0`
Output	`5`
Explanation	*The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.*

Example 3

Input	`matrix = [[904]], target = 0`
Output	`0`

CONSTRAINTS

- 1 <= matrix.length <= 100
- 1 <= matrix[0].length <= 100
- 1000 <= matrix[i][j] <= 1000
- 10^8 <= target <= 10^8

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count all submatrices (r1,c1,r2,c2) whose elements sum to target. Right?

Extend #560 Subarray Sum Equals K to 2D."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ranges) in ranges)s:
    sum its cells
    count those equal to target
```

TIME	SPACE
O(m^2 * n^2 * m * n)	O(1)

WHY IT IS SLOW

Re-sums every submatrix. For each pair of rows compress the columns into a 1D array and use the prefix-sum hash map from Subarray Sum Equals K: O(m^2 * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Fix top/bottom rows, apply 1-D subarray sum on compressed columns — O(m²n). Walk-through:"

2-D prefix sums for O(1) column-range sums.

For each pair (r1, r2) of rows (n² pairs):

For each column j: col_sum[j] = prefix sum of rows r1..r2 in column j.

This gives a 1-D array of column sums.

Apply #560 (subarray sum equals target) on col_sum:

running_prefix_counts[0]=1. For each cumulative sum: count += prefix_counts[cum-target].

WHY reduce to 1-D: fixing rows r1..r2 collapses the 2-D problem to a 1-D subarray sum.

Time: O(m²n) — O(m²) row pairs × O(n) per pair. Space: O(mn) prefix + O(n) column.

PHASE 4 CLEAN CODE

Apply #560 on col_sum

```python
class _1074_SubmatricesSumTarget:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        m, n  = len(matrix), len(matrix[0])
        count = 0
        for r1 in range(m):
            col_sum = [0] * n
            for r2 in range(r1, m):
                for c in range(n):
                    col_sum[c] += matrix[r2][c]
```

```
            prefix = Counter({0: 1})
            running = 0
            for s in col_sum:
                running += s
                count   += prefix[running - target]
                prefix[running] += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace numSubmatrixSumTarget([[0,1,0],[1,1,1],[0,1,0]], target=0)."

Fix top/bottom row pair, compute column prefix sums, then find subarrays summing to target

For rows 0-0: col sums=[0,1,0]. Find subarrays=0: [0],[0],[0,1,0]=1? 0->1 match, prefix{0:1,1:1,1:1}...

Count subarrays with sum=target=0: count prefix map

Accumulate over all (r1,r2) pairs -> total count ok

Trace: answer=4 for given example

WHY reduce to 1D: fix row boundaries, collapse to column prefix sums -> O(m²*n) with hash map.

Same as 'subarray sum equals k' applied to each column-compressed row pair.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m² * n)	**O(n)** This is literally #560 applied inside a 2D row-fixing loop

Name that connection — it shows pattern transfer.

Given more time I'd ask: count submatrices summing to at most target?

Use #862 (Shortest Subarray with Sum at Least K) approach — monotone deque instead of hash map.

IN YOUR APP | Expose | Editorial | Community

HARD | Matrix | LeetCode #3651 | 4 of 4 | 4/4

Minimum Cost Path with Teleportations

QUESTION

You are given a m x n 2D integer array grid and an integer k. You start at the top-left cell (0, 0) and your goal is to reach the bottomright cell (m - 1, n - 1).

There are two types of moves available:

CONSTRAINTS

- Normal move: You can move right or down from your current cell (i, j), i.e. you can move to (i, j + 1) (right) or (i + 1, j) (down). The cost is the value of the destination cell.
- Teleportation: You can teleport from any cell (i, j), to any cell (x, y) such that grid[x][y] <= grid[i][j]; the cost of this move is 0. You may teleport at most k times.

Return the minimum total cost to reach cell (m - 1, n - 1) from (0, 0).

Example 1

Input	`grid = [[1,3,3],[2,5,4],[4,3,5]], k = 2`
Output	`7`
Explanation	*Initially we are at (0, 0) and cost is 0. The minimum cost to reach bottom-right cell is 7.*

Example 2

Input	`grid = [[1,2],[2,3],[3,4]], k = 1`
Output	`9`
Explanation	*Initially we are at (0, 0) and cost is 0. The minimum cost to reach bottom-right cell is 9.*

- 2 <= m, n <= 80
- m == grid.length
- n == grid[i].length
- 0 <= grid[i][j] <= 10^4
- 0 <= k <= 10

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Grid with edge costs and teleport options; find min cost to reach bottom-right. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
dp over (cell, teleports used): from every cell move
    right or down at the destination's cost, or
    teleport to every cell whose value is not larger
    at cost 0 (at most k times)
take the cheapest cost to the corner
```

TIME	SPACE
O(k * (m * n)^2)	O(m * n)

WHY IT IS SLOW

Tries every teleport target from every cell. Process teleport layers; in each layer sort the cells by value and keep a running minimum of the best cost so far over cells of value >= v, so a teleport lookup is O(1): O(k * m * n log(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Dijkstra with teleport edges — O((V+E) log V). Walk-through:"

Standard Dijkstra but add teleport edges: from each room (r,c), you may teleport to any

previously-visited room for a fixed cost (or 0 depending on problem variant).

Model teleport as edges from each node to all others with teleport cost.

Since teleport destinations are ALL prior nodes, expand edge set appropriately.

State = (cost, r, c). Min-heap expansion.

For teleport: when processing (cost, r, c), add edges to teleport destinations.

WHY Dijkstra: non-uniform edge costs (step vs teleport) require shortest-path algorithm.

Time: O((V+E) log V) where E includes teleport edges. Space: O(V+E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _3651_MinCostPathWithTeleportations:
    def minCost(self, grid):
        import heapq
        m,n=len(grid),len(grid[0])
        dist=[[float('inf')]*n for _ in range(m)]; dist[0]
[0]=0
        heap=[(0,0,0)]
        while heap:
            d,r,c=heapq.heappop(heap)
            if d>dist[r][c]: continue
            for dr,dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr,nc=r+dr,c+dc
                if 0<=nr<m and 0<=nc<n:
                    nd=d+grid[nr][nc]
```

```
                    if nd<dist[nr][nc]: dist[nr][nc]=nd; h
eapq.heappush(heap,(nd,nr,nc))
        return dist[m-1][n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace minCostTeleport(3x3 grid, cost=1/step, teleporters=[(1,1)])."

State: (row, col, mask) where mask=0 or 1 (teleporter at (1,1) used or not)

Dijkstra heap: [(0,0,0,0b0)] (cost,r,c,mask)

Pop(0,0,0,0): move right->(0,1,0b0,cost=1); move down->(1,0,0b0,cost=1); teleport to (1,1) free->(1,1,0b1,cost=0)

Pop(0,1,1,0b1): from (1,1) with teleporter used: move right->(1,2,0b1,1); move down->(2,1,0b1,1)

Pop(1,0,1,0b0): move to (0,1),(1,1) with cost 2; or teleport (mask already 0) to (1,1) cost=1+0=1 -> but (1,1,0b1) already cost=0

Expand until reach (2,2,any): min cost = 2 (teleport (0,0)->(1,1) free, then 2 steps) ok

WHY bitmask: each unique (r,c,mask) is a distinct state; prevents teleporter reuse.

State count: m*n*2^k — feasible for small k (typically <=15 teleporters).

dp[r][c][mask]=min cost to reach (r,c) with teleporter mask. dp[0][0][0b0]=0. dp[i][j][mask]=min(dp[i-1][j][mask]+1,dp[i][j-1][mask]+1,...). Dijkstra over (r,c,mask) states.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn log mn)	**O(n)**
Add teleport edges as (0, teleport_dest) to the heap."	DP table, reducible to O(1) rolling window for 1-D recurrences

Follow-up: what if some teleportations are one-way only? Convert them to directed edges and re-run Dijkstra — the key invariant (monotone cost) still holds.